INFORME TÉCNICO INTEGRAL

SISTEMA DE GESTIÓN: BODEGAS DEL ATUEL

Autores: Lucas Bordón, Matias Bordón, Juan Andrés López

1. Introducción y Alcance

1.1. Introducción

El presente informe documenta el diseño, desarrollo e implementación del nuevo sistema de gestión integral para Bodegas del Atuel S.A.. Ante la creciente complejidad de las operaciones de la bodega —que abarcan desde la producción agrícola y el control de calidad de la vid, hasta la distribución logística y comercialización de los vinos—, surgió la necesidad crítica de centralizar la información en una arquitectura de software robusta, eficiente y escalable.

El núcleo de este proyecto se apoya en una base de datos relacional avanzada (PostgreSQL), bajo la filosofía de delegar la mayor cantidad posible de validaciones, restricciones de integridad estructural y reglas de negocio directamente en el motor de base de datos (mediante el uso de constraints, triggers y bloqueos transaccionales). Esto garantiza un entorno seguro, consistente y de alto rendimiento capaz de soportar operaciones concurrentes.

1.2. Alcance del Proyecto

El alcance del presente sistema comprende el diseño del modelo de datos normalizado (3FN), la optimización del rendimiento mediante índices, la carga masiva de datos transaccionales de prueba y el desarrollo de una API Backend para la orquestación de las operaciones. El sistema cubre los siguientes módulos funcionales:

- Módulo Agronómico y Geográfico: Gestión de la jerarquía territorial (provincias, departamentos, localidades), administración de fincas y parcelas, y el registro de lotes de uva por campaña junto con sus análisis de calidad enológicos (grados Brix, pH, acidez).
- Módulo de Catálogo y Enología: Administración del catálogo comercial de vinos, control estricto de los porcentajes de composición por cepa (garantizando cortes exactos del 100%) y gestión de maridajes.
- Módulo Comercial y Logístico: Gestión de una base de clientes segmentada (socios, distribuidores y exportadores con control de permisos y aduanas), facturación con cálculo dinámico de descuentos, y un riguroso sistema logístico que controla los ingresos al depósito central y las transferencias de inventario hacia las sucursales (salas de venta) mediante una máquina de estados.

1.3. Fuera de Alcance

Para esta iteración del proyecto, quedan excluidos del alcance:

- La integración con pasarelas de pago financieras, sistemas bancarios o de facturación electrónica externa (AFIP).
- El módulo de liquidación de sueldos y recursos humanos avanzados (limitándose únicamente a la asignación de empleados a salas de venta).
- El desarrollo completo de la interfaz gráfica de usuario final (Frontend), enfocando los esfuerzos de este documento en la arquitectura de datos, la API y la lógica de negocio.

2. Modelo de Datos

2.1. Diagrama Entidad-Relación (DER) y Esquema Relacional

El esquema relacional de la base de datos ha sido particionado lógicamente en tres módulos principales para facilitar su comprensión y mantenimiento:

- Módulo Agronómico y Geográfico: Estructura la jerarquía productiva (Fincas, Parcelas, Lotes de Uva y Análisis de Calidad) y la jerarquía territorial (Provincias, Departamentos, Localidades).
- Módulo de Catálogo y Enología: Modela el núcleo de productos, detallando las Bodegas, Vinos, Cepas (con sus porcentajes de corte) y Maridajes sugeridos.
- Módulo Comercial y Logístico: Define la red de Salas de Venta, la tipificación de Clientes (Socios, Distribuidores, Exportadores), la facturación (Comprobantes y renglones), los Ingresos de mercadería y las Transferencias entre sucursales.

2.2. Normalización

El modelo de datos ha sido diseñado respetando los principios de la Tercera Forma Normal (3FN). Esto garantiza que:

- Todos los atributos atómicos dependen de la clave primaria completa (cumplimiento de la 1FN y 2FN).
- No existen dependencias transitivas entre atributos no clave (cumplimiento de la 3FN). Por ejemplo, la estructura geográfica fue aislada en las entidades Provincia, Departamento y Localidad en lugar de guardar datos en texto plano dentro de la tabla Sala_Venta o Cliente.
- Las relaciones de muchos a muchos (N:M) fueron resueltas mediante tablas asociativas con claves primarias compuestas (ej. Vino_Cepa, Vino_Maridaje, Vino_Comprobante).

2.3. Diccionario de Datos

Para asegurar la consistencia semántica, los tipos de datos y las restricciones de dominio, se utilizó una hoja de cálculo estructurada como herramienta base para la creación y mantenimiento del diccionario de datos del proyecto.

Localidad

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_localidad	INT	NO	-	PK	Identificador numérico único de la localidad.
nombre	VARCHAR	NO	50	-	Nombre de la localidad para indentificación
id_provincia	INT	NO	50	FK	Identificador número único de la provincia.
codigo_postal	VARCHAR	NO	10	-	Codigo postal de la localidad
encargado_localidad	VARCHAR	NO	100		CUIT del encargado de la localidad.

Provincia

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_provincia	INT	NO	-	PK	Identificador numérico único de la provincia.
nombre	VARCHAR	NO	30	-	Nombre de provincia.

Sala de venta

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_sala	INT	NO	-	PK	Identificador numérico único de la sala de venta.
calle	VARCHAR	NO	100	-	Nombre de la calle donde se ubica la sala.
numero	INT	NO	-	-	Número de la dirección de la sala de venta.
dia_cierre_semanal	VARCHAR	NO	15	-	Día de la semana en que la sala no opera.
id_localidad	INT	NO	-	FK	Identificador de la localidad a la que pertenece la sala.

Empleado

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
CUIT	CHAR	NO	11	PK	Clave Única de Identificación Tributaria del empleado (sin guiones).
nombre	VARCHAR	NO	100	-	Nombre completo del empleado.
fecha_ingreso	DATE	NO	-	-	Fecha en la que el empleado comenzó a trabajar.
salario	DECIMAL	NO	10,2	CHECK (> 0)	Sueldo actual del empleado.
id_sala	INT	NO	-	FK	Identificador de la sala de venta donde trabaja el empleado.

Finca

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_finca	INT	NO	-	PK	Identificador numérico único de la finca.
nombre	VARCHAR	NO	100	-	Nombre de la finca.
superficie_total	DECIMAL	NO	10,2	CHECK (> 0)	Superficie total que abarca la finca en hectareas.
id_departamento	VARCHAR	NO	100	FK	Código del departamento donde se localiza la finca.

Departamento

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_departamento	INT	NO	-	PK	Identificador número único del departamento.
id_provincia	INT	NO	-	FK	Identificador número único de la provincia.
nombre_departamento	VARCHAR	NO	30	-	Nombre del departamento.

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
nro_parcela	INT	NO	-	PK	Identificador numérico único de la parcela.
superficie	DECIMAL	NO	10,2	CHECK (> 0)	Superficie total que abarca la parcela en hectareas.
ano_plantación	INT	NO	-	CHECK (> 0)	Año en el que se
id_finca	INT	NO	-	PK	Identificador de la finca a la que pertenece la parcela.
id_cepa	VARCHAR	NO	100	FK	Identificador de la cepa plantada en la finca.

Cepa

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_cepa	INT	NO	-	PK	Identificador numérico único de la cepa.
nombre	VARCHAR	NO	50	-	Nombre de la cepa.
tipo	ENUM	NO	-	('tinta', 'blanca')	Puede ser del tipo "tinta" o del tipo "blanca".

Lote uva

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
nro_lote	INT	NO	-	PK	Identificador numérico único del lote de uva.
nro_parcela	INT	NO	-	FK	Identificador de la parcela a la que pertence el lote de uva.
campana	INT	NO	-	-	Año de cosecha del lote de uva.
fecha_cosecha	DATE	NO	-	-	Fecha especifica de la cosecha.
kilos_cosechados	DECIMAL	NO	10,2	CHECK (> 0)	Peso de la cosecha en kg.
id_finca	INT	NO	-	FK	Identificador de la finca a la que pertenece el lote de uva.
id_bodega	INT	NO	-	FK	Identificador de la bodega destino del lote.

Análisis de calidad

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_analisis	INT	NO	-	PK	Identificador numérico único del análisis de calidad .
fecha	DATE	NO		-	Fecha en que se efectuó el análisis.
ph	DECIMAL	NO	10,2	CHECK (> 0)	Nivel de acidez o alcalinidad de la muestra.
acidez_total	DECIMAL	NO	10,2	CHECK (> 0)	Acidez total medida en la muestra (g/L).
g_brix	DECIMAL	NO	10,2	CHECK (> 0)	Concentración de azúcares presentes en la uva.
nro_lote	INT	NO	-	FK	Número del lote evaluado.

Bodega

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
cuit	CHAR	NO	11	PK	Identificador único Tributario que identifica fiscalmente a la bodega.
razon_social	VARCHAR	NO	100	-	Nombre legal o denominación registrada de la bodega ante los organismos correspondientes.
domicilio	VARCHAR	NO	250	-	Dirección física donde se encuentra ubicada la bodega o su sede principal.

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo_vino	VARCHAR	NO	30	PK	Identificador único asignado a cada vino elaborado o comercializado por la bodega.
nombre	VARCHAR	NO	100	-	Nombre comercial con el que se identifica el vino en el mercado.
tipo	ENUM	NO	-	('tinto', 'blanco', 'rosado', 'espumante')	Clasificación del vino según sus características (tinto, blanco, rosado o espumante).
anada	INT	NO	-	CHECK (> 0)	Año de cosecha de las uvas utilizadas para la elaboración del vino.
volumen	INT	NO	-	CHECK (> 0)	Cantidad de producto contenida en el envase (ml).
graduacion	DECIMAL	NO	4,2	CHECK (> =0)	Porcentaje de alcohol que contiene el vino (% vol.).
precio	DECIMAL	NO	4,2	CHECK(>0)	Precio unitario del vino.
CUIT	CHAR	NO	11	FK	CUIT de la bodega productora o responsable del vino.

Vino_Cepa

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo_vino	VARCHAR	NO	30	PK, FK	Referencia al vino específico.
id_cepa	INT	NO	-	PK, FK	Referencia a la cepa utilizada en el vino.
porcentaje	DECIMAL	NO	5,2	CHECK (> 0)	Porcentaje de participación de la cepa en el vino.

Maridaje

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_maridaje	INT	NO	-	PK	Identificador numérico único de cada recomendación de maridaje.
nombre	VARCHAR	NO	100	-	Nombre o descripción del alimento, plato o categoría gastronómica recomendada para acompañar el vino.
temperatura	DECIMAL	NO	4,1	-	Temperatura en grados celsius (°C) recomendada para servir el vino y obtener sus mejores características organolépticas.

Vino_Maridaje

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo_vino	VARCHAR	NO	30	PK, FK	Referencia al vino específico.
id_maridaje	INT	NO	-	PK, FK	Referencia al maridaje específico.

Cliente

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_cliente	INT	NO	-	PK	Identificador numérico único del cliente.
nombre_razon_social	VARCHAR	NO	150	-	Nombre personal o denominación legal de la empresa.
tipo_doc	VARCHAR	NO	10	-	Categoría del documento presentado (ej. DNI, CUIT).
nro_doc	VARCHAR	NO	15	UNIQUE	Número identificatorio asociado al tipo de documento.
direccion	VARCHAR	NO	200	-	Domicilio físico o legal declarado por el cliente.
id_localidad	INT	NO	-	FK	Referencia de la localidad de residencia.
fecha_alta	DATE	NO		-	Fecha de registro del cliente en el sistema de la bodega.

Socio del club

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_cliente	INT	NO	-	PK, FK	Referencia del cliente.

Distribuidores

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_cliente	INT	NO	-	PK, FK	Referencia del cliente específico.
zona_cobertura	VARCHAR	NO	100	-	Región o área geográfica donde opera.
nro_habitacion	VARCHAR	NO	50	UNIQUE	Número de registro o habilitación comercial.

Exportadores

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_cliente	INT	NO	-	PK, FK	Referencia del cliente específico.
pais_destino	VARCHAR	NO	50	-	País principal de exportación.

Permiso de Exportación

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_permiso	VARCHAR	NO	50	PK	Código único del permiso de exportación.
id_cliente	INT	NO	-	FK	Referencia del cliente específico.
fecha_emision	DATE	NO	-	-	Fecha de inicio de validez del permiso.
fecha_vencimiento	DATE	NO	-	CHECK (> fecha_emision)	Fecha de caducidad del permiso.

Comprobante

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
numero	INT	NO	-	PK	Identificador numérico único del comprobante.
fecha	DATE	NO	-	-	Fecha en la que se efectuó el comprobante.
id_cliente	INT	NO	-	FK	Referencia del cliente específico.
id_sala	INT	NO	-	FK	Referencia de la sala específica en la que se realizo.
subtotal	DECIMAL	NO	10,2	CHECK (> =0)	Precio total sin descuento.
descuento	DECIMAL	NO	10,2	DEFAULT 0, CHECK (>= 0)	Porcentaje de bonificación aplicable a sus compras.
total	DECIMAL	NO	10,2	CHECK (> =0)	Precio total final con o sin descuento.

Vino_Comprobante (Renglón de comprobante)

Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
nro_comprobante	INT	NO	-	PK, FK	Referencia del comprobante al que corresponde.
cod_vino	VARCHAR	NO	30	PK, FK	Referencia al vino específico.
cantidad	INT	NO	-	CHECK (> 0)	Cantidad de unidades del producto que se está comprando.
precio_unitario	DECIMAL	NO	10,2	CHECK (> =0)	Precio unitario del producto.
subtotal	DECIMAL	NO	10,2	CHECK (> =0)	Precio total sin descuento.
descuento	DECIMAL	NO	5,2	DEFAULT 0, CHECK (>= 0)	Porcentaje de bonificación aplicable a sus compras.

Transportista					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo	VARCHAR	NO	30	PK	Identificador único de la empresa transportista.
razon_social	VARCHAR	NO	100	-	Nombre legal o denominación registrada de la empresa ante los organismos correspondientes.
CUIT	CHAR	NO	11	UNIQUE	Identificador único Tributario que identifica fiscalmente a la empresa transportista.
email	VARCHAR	NO	255	-	Correo electrónico de contacto.
telefono	VARCHAR	NO	20		Número de teléfono de contacto
domicilio	VARCHAR	NO	250	-	Dirección física donde se encuentra ubicada la empresa o su sede principal.

Transportista_Localidad					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo	VARCHAR	NO	30	PK, FK	Referencia a la empresa de transporte correspondiente.
id_localidad	INT	NO	-	PK, FK	Referencia a la empresa de transporte correspondiente.

Ingreso					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo	INT	NO	-	PK	Identificador único del remito de ingreso.
fecha	DATE	NO	-	-	Fecha en la que ingresa el stock al depósito.
estado	ENUM	NO	-	('en confección', 'terminado', 'procesado')	Estado en el que se encuentra el ingreso en un determinado momento.
cod_transportista	VARCHAR	NO	30	FK	Referencia a la empresa del transporte.
cuit_bodega	CHAR	NO	11	FK	Referencia a la bodega de origen.

Ingreso_Vino					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo_ingreso	INT	NO	-	PK, FK	Referencia al ingreso correspondiente.
codigo_vino	VARCHAR	NO	30	PK, FK	Referencia al vino ingresante.
cantidad	INT	NO	-	CHECK (> 0)	Cantidad de producto que ingresa.

Transferencia					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
nro_transferencia	INT	NO	-	PK	Identificador numérico único de la transferencia de producto.
id_sala	INT	NO	-	FK	Referencia a la sala solicitante.
fecha_confeccion	DATE	NO	-	DEFAULT CURRENT_DATE	Fecha en la que se emitió la solicitud.
estado	ENUM	NO	-	('en confección', 'terminado', 'procesado')	Estado en el que se encuentra la transferencia en un determinado momento.

Transferencia_Vino					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
nro_transferencia	INT	NO	-	PK, FK	Referencia a la transferencia a una sala.
codigo_vino	VARCHAR	NO	30	PK, FK	Referencia al vino transferido.
cantidad	INT	NO	-	CHECK (> 0)	Cantidad de producto transferido.

Stock de sala					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_sala	INT	NO	-	PK, FK	Referencia a la sala correspondiente.
codigo_vino	VARCHAR	NO	30	PK, FK	Referencia al vino almacenado.
cantidad	INT	NO	-	CHECK (>=0)	Cantidad de producto almacenado.

Stock de depósito					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
codigo_vino	VARCHAR	NO	30	PK, FK	Referencia al vino almacenado.
cantidad	INT	NO	-	CHECK (>=0)	Cantidad de producto almacenado.

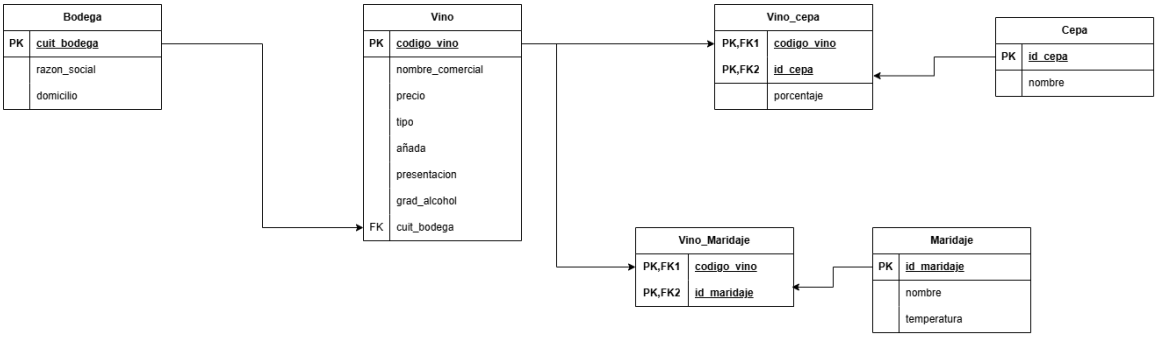
Auditoria de precio					
Nombre del campo	Tipo de dato	Admite null	Tamaño	Restricciones	Descripción
id_auditoria	INT	NO	-	PK	Identificador numérico único de la auditoría.
codigo_vino	VARCHAR	NO	30	FK	Referencia al vino correspondiente.
precio_anterior	DECIMAL	NO	10,2	CHECK (> =0)	Valor anterior de venta del vino.
precio_nuevo	DECIMAL	NO	10,2	CHECK (> =0)	Valor nuevo de venta del vino.
fecha_hora	DATETIME	NO		DEFAULT CURRENT_TIMESTAMP	Fecha y hora de la auditoría.
usuario	CHAR	NO	11	FK	Referencia a la Identificación Tributaria del empleado (CUIT, sin guiones) que realiza la actualización de precio.

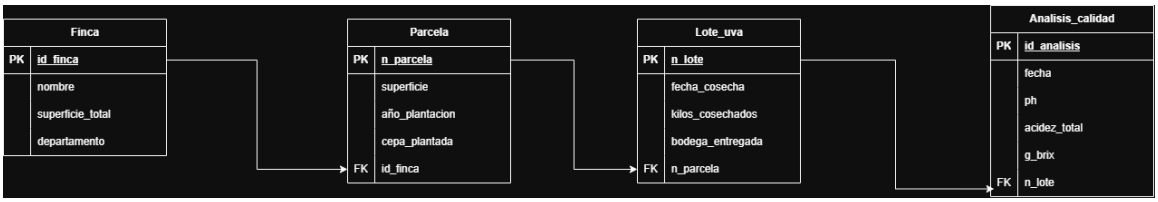
2.4. Decisiones de Diseño y Arquitectura

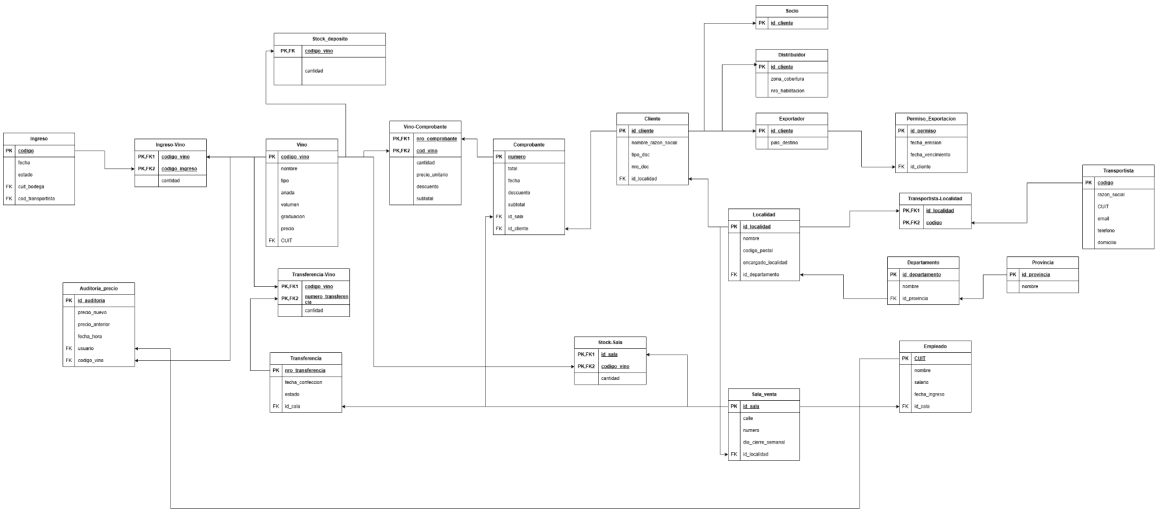
Durante el modelado, se evaluaron diversas alternativas arquitectónicas, adoptando las siguientes decisiones fundamentadas en las reglas del negocio y el rendimiento operativo:

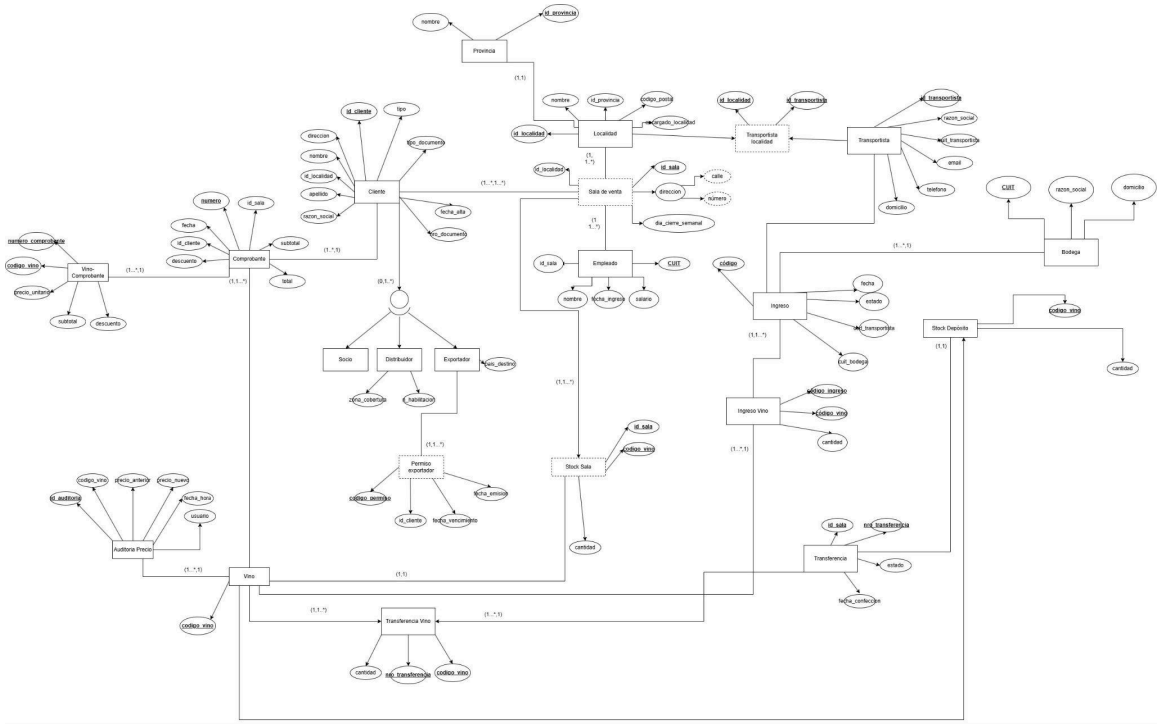
- Clave Primaria Subrogada en Localidades: Se decidió implementar un identificador subrogado (id_localidad) como Clave Primaria de la entidad Localidad, descartando el uso del código postal. Esta decisión previene fallas estructurales graves (cambios en cascada) ante eventuales modificaciones en la normativa de códigos postales a nivel gubernamental.
- Mapeo Unificado de Clientes: En la entidad Cliente, se unificaron los atributos de denominación en un único campo llamado nombre_razon_social. Si se hubieran modelado por separado (Nombre vs. Razón Social), se habrían generado registros incompletos sistemáticamente con valores NULL (dependiendo de si el cliente era persona física o entidad jurídica).

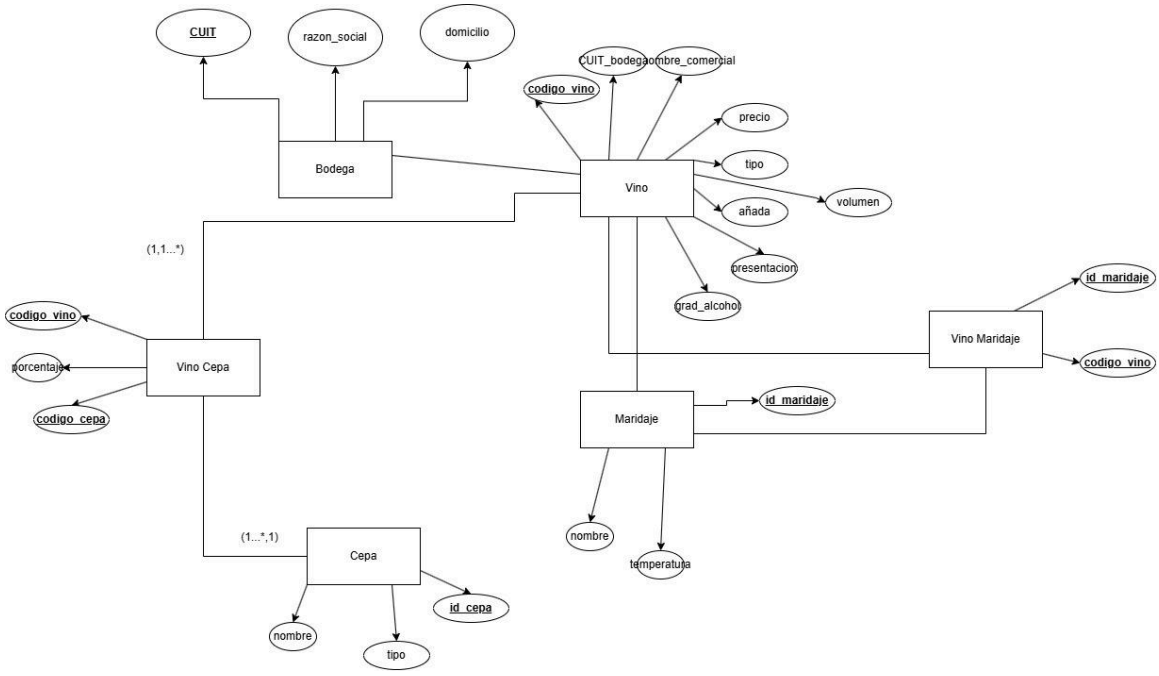
- Flexibilidad en los Descuentos (Renglones vs. Cabecera): Aunque en un diseño estrictamente normalizado el descuento general de cliente debería residir en la cabecera (Comprobante), se optó por incorporar un atributo de descuento directamente en los renglones (Vino_Comprobante). Esto otorga flexibilidad al sistema para aplicar bonificaciones específicas por producto o promoción (ej. 10% de descuento en Malbec, pero 0% en Chardonnay en la misma compra).
- Manejo de Atributos Derivados para Rendimiento: Se tomó la decisión consciente de persistir físicamente los campos subtotal y total en las tablas transaccionales, a pesar de ser teóricamente atributos derivados (calculables como cantidad * precio - descuento). Esta redundancia controlada simplifica el desarrollo de reportes analíticos y mejora drásticamente el rendimiento de lectura del motor de base de datos.
- Descuento de Cliente Dinámico: A diferencia del subtotal, el porcentaje de descuento base de un cliente se dejó como un campo calculado dinámicamente y no persistido en el comprobante. Dado que las políticas comerciales varían con el tiempo, actualizar masivamente registros individuales ante un cambio de política resultaría contraproducente y costoso para el motor.

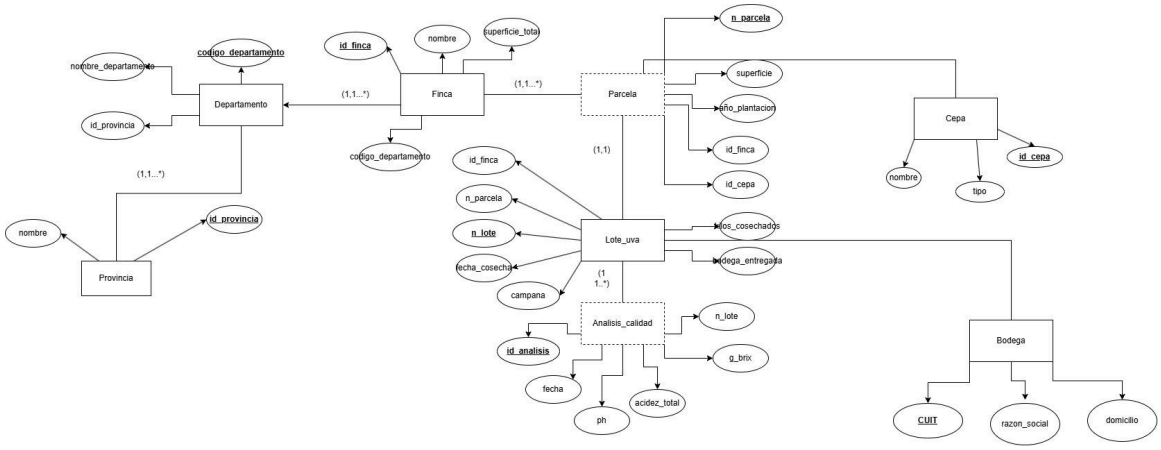

Bodega
PK cuit_bodega
razon_social
domicilio
Vino
PK codigo_vino
nombre_comercial
precio
tipo
añada
presentacion
grad_alcohol
FK cuit_bodega
Vino_cepa
PK,FK1 codigo_vino
PK,FK2 id_cepa
porcentaje
Cepa
PK id_cepa
nombre
Vino_Maridaje
PK,FK1 codigo_vino
PK,FK2 id_maridaje
Maridaje
PK id_maridaje
nombre
temperatura


Finca
PK id_finca
nombre
superficie_total
departamento
Parcela
PK n_parcela
superficie
año_plantacion
cepa_plantada
FK id_finca
Lote_uva
PK n_lote
fecha_cosecha
kilos_cosechados
bodega_entregada
FK n_parcela
Analisis_calidad
PK id_analisis
fecha
ph
acidez_total
g_brix
FK n_lote


Socio
Distribuidor
Exportador
Stock_deposito
Ingreso
Ingreso Vino
Vino
Vino-Comprobante
Comprobante
Cliente
Permiso_Exportacion
Transportista
Transportista-Localidad
Localidad
Departamento
Provincia
Auditoria_precio
Transferencia Vino
Transferencia
Stock_Sala
Sala_venta
Empleado


Provincia
Localidad
Transportista
Transportista localidad
Sala de venta
Cliente
Comprobante
Vino-Comprobante
Empleado
Bodega
Ingreso
Ingreso Vino
Stock Deposito
Socio
Distribuidor
Exportador
Permiso exportador
Stock Sala
Auditoria Precio
Vino
Transferencia
Transferencia Vino


CUIT
razon_social
domicilio
Bodega
CUIT_bodega
nombre_comercial
codigo_vino
precio
tipo
volumen
añada
presentacion
grad_alcohol
Vino
(1,1...*)
Vino Cepa
codigo_vino
porcentaje
codigo_cepa
Vino Maridaje
id_maridaje
codigo_vino
Maridaje
id_maridaje
nombre
temperatura
(1...*,1)
Cepa
id_cepa
nombre
tipo



3. Reglas de Negocio Implementadas

3.1. Guía de Arquitectura: ¿Dónde va cada regla?

Para el sistema de Bodegas del Atuel, se adoptó una arquitectura de "Soberanía de Datos" o "Base de Datos Gruesa", definiendo criterios estrictos sobre qué lógica debe ser implementada directamente en el motor relacional (PostgreSQL) y qué responsabilidades pertenecen al Backend (FastAPI).

- Motor de Base de Datos (Constraints y Triggers): Se delegaron al motor todas aquellas reglas de negocio que garantizan la integridad absoluta de la información, independientemente de la aplicación cliente que se conecte.
- Constraints (CHECK, UNIQUE, NOT NULL, FKs): Utilizadas para validaciones atómicas de fila (ej. precios positivos, graduación alcohólica entre 5% y 20%, que una parcela solo tenga una cosecha por campaña).
- Triggers en PL/pgSQL: Empleados para lógica transversal, máquinas de estados y auditorías. Por ejemplo, automatizar el registro en la tabla de auditoría ante cambios de precio, forzar la fecha_confeccion automática evitando ingresos manuales.
- Backend / API (Código de Aplicación): Se reserva para la orquestación, validación sintáctica de los payloads entrantes, autorización, y manejo de excepciones (capturar los errores generados por los Triggers o Constraints de la base de datos y traducirlos a códigos HTTP 400/409 comprensibles para el usuario).

3.2. Tabla de Trazabilidad: Regla → Mecanismo → Prueba

Regla de Negocio (RN)	**Mecanismo de Implementación**	**Método de Prueba / Verificación**
RN-01: Precio de un vino único y mayor a cero.	Atributo precio en vino con restricción CHECK (precio > 0).	Inserción y actualización de vinos con precio positivo.

RN-02 y RN-03: Sala pertenece a 1 localidad. Empleado trabaja en 1 sala.	FK id_localidad en sala_venta y FK id_sala en empleado (NOT NULL).	Cada sala queda asociada a una localidad y el empleado a una única sala.
RN-04: Parcela con una sola cepa y máximo una cosecha por campaña.	FK id_cepa en parcela y restricción UNIQUE (id_finca, nro_parcela, campana) en lote_uva.	Intento de registrar dos cosechas en la misma parcela/campaña es rechazado.
RN-05: Porcentajes de corte de vino suman 100%.	Constraint Trigger diferido sobre tabla vino_cepa.	Se valida la suma total de porcentajes al finalizar la transacción (COMMIT).
RN-06: Descuento por tipo de cliente (Socio: 10%, Distribuidor: 20%, Exportador: 30%).	Función de cálculo de descuentos aplicada automáticamente al comprobante.	Se comprueba que el descuento aplicado coincida exactamente con el tipo de cliente.
RN-07: Exportador con vencimiento de permiso posterior a su emisión.	Tabla permiso_exportacion con restricción CHECK (fecha_vencimiento > fecha_emision).	Todo permiso ingresado posee fechas cronológicamente válidas.
RN-08: Total de comprobante es la suma de renglones con descuento.	Trigger de cálculo automático y actualización de atributos subtotal y total.	Importes se recalculan instantáneamente al modificar cualquier renglón.
RN-09: Stock nunca negativo, no vender más de lo disponible en sala.	Trigger de validación previo a la venta y restricciones CHECK de stock no negativo.	Venta rechazada al intentar superar el stock físico disponible.
RN-10: Solo ingresos "terminados" pasan a "procesado" impactando en depósito.	Tipo ENUM para estados, Trigger de control y procedimiento almacenado procesar_ingreso.	Bloqueo de procesamiento a ingresos en confección y bloqueo de retrocesos de estado.
RN-11: Transferencias siguen estados de ingresos. Fecha actual e inmutable.	Triggers BEFORE INSERT y BEFORE UPDATE para asignar fecha de confección automática y bloquear ediciones.	La fecha se asigna por sistema, ignorando ingresos manuales y previniendo alteraciones.
RN-12: Bloqueo de renglones si ingreso/transferencia no está "en_confeccion".	Trigger de bloqueo de edición de renglones atado al estado del documento de cabecera.	Solo los documentos en estado en_confeccion permiten agregar, modificar o borrar líneas.
RN-13: Cambio de precio de vino deja registro en auditoría.	Trigger AFTER UPDATE sobre vino insertando en la tabla auditoria_precio.	Se verifica el registro automático de fecha/hora, usuario, precio anterior y nuevo.
RN-14: Prohibido eliminar transportista, bodega o vino con movimientos.	Claves foráneas (FK) con restricción ON DELETE RESTRICT.	Eliminación rechazada por la base de datos al existir referencias activas.
RN-15: Cantidades/precios positivos y graduación alcohólica válida.	Restricciones CHECK (positivos y graduación BETWEEN 5 AND 20).	Motor rechaza ingresos fuera de parámetros directamente a nivel de fila.

3.3. Demostración Práctica de Concurrencia (RN-09)

Para asegurar el estricto cumplimiento de la RN-09 (Control de stock no negativo) bajo un entorno multiusuario, se implementó un sistema de bloqueos pesimistas. A continuación se demuestra cómo la cláusula FOR UPDATE evita condiciones de carrera (race conditions) durante la comercialización o movimiento de inventario.

Para esta prueba, se simulan dos sesiones (Sesión A y Sesión B) operando simultáneamente sobre el mismo registro de stock en PostgreSQL.

Estado Inicial: Se parte de un escenario donde el producto VINO001 posee un stock inicial de 1000 unidades en el depósito.

```
SELECT * FROM stock_deposito WHERE codigo_vino =
'VINO001'; -- Resultado: 1000
```

Paso 1: Intervención de la Sesión A. La Sesión A inicia una transacción y consulta el stock aplicando el bloqueo explícito.

```
BEGIN; SELECT cantidad FROM stock_deposito WHERE
codigo_vino = 'VINO001' FOR UPDATE; -- La fila
queda bloqueada exclusivamente para la Sesión A.
```

Paso 2: Intervención de la Sesión B (Concurrente). Mientras la transacción de la Sesión A sigue abierta, la Sesión B intenta acceder al mismo registro.

```
BEGIN; SELECT cantidad FROM stock_deposito WHERE
codigo_vino = 'VINO001' FOR UPDATE; -- La consulta
queda en estado de espera.
```

Paso 3: Actualización y liberación por parte de la Sesión A. La Sesión A procede a descontar 100 unidades de stock y confirma la transacción.

```
UPDATE stock_deposito SET cantidad = cantidad - 100
WHERE codigo_vino = 'VINO001'; COMMIT; -- La fila
queda liberada. El stock actualizado pasa a ser
900.
```

Paso 4: Resolución de la Sesión B. Automáticamente al liberarse el bloqueo de la Sesión A, el motor continúa la consulta de la Sesión B, la cual obtiene el valor real y actualizado.

```
-- Resultado automático devuelto en la Sesión B:
900 COMMIT;
```

Conclusión: Sin Bloqueo vs. Con Bloqueo

- Riesgo sin FOR UPDATE: Ambas sesiones leen simultáneamente el valor inicial de 1000. Ambas descuentan 100 de forma independiente sobre un valor obsoleto, generando riesgo de inconsistencias graves y stock sobrevendido.
- Consistencia con FOR UPDATE: La Sesión A bloquea la fila. La Sesión B es forzada a aguardar su turno. Cuando la Sesión A actualiza y confirma, la Sesión B recién puede acceder a la fila obteniendo el stock real actualizado (900), garantizando integridad ACID estricta.

4. Generación y Carga Masiva de Datos

Para cargar los datos, decidimos mantener desactivados los triggers.

Paso 1: Carga masiva (con registros de tiempo). Primero validamos la ejecución de nuestro esquema.

Luego de levantar el contenedor usando nuestro docker compose, efectuamos este comando y observamos el resultado:

```
sudo docker exec -i bodegas_db psql -U postgres -d
bodegas_atuel < db/migrations/001_esquema.sql
```

```
DROP SCHEMA
CREATE SCHEMA
GRANT
GRANT
CREATE TYPE
CREATE TYPE
CREATE TYPE
CREATE TYPE
CREATE DOMAIN
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

Coincide con nuestro esquema y su orden de ejecución. No saltó ningún error. El esquema está bien estructurado.

Ahora procede la carga masiva.

Con el comando `time uv run db/seed/script_carga.py` observaremos si se ejecuta correctamente nuestro script de carga.

```
Iniciando carga de datos...
Catálogo cepa cargado correctamente.
Catálogo maridaje cargado correctamente.
Catálogo provincia cargado correctamente.
Catálogo departamento cargado correctamente.
Catálogo localidad cargado correctamente.
Generando datos masivos con Faker (Esto tomará unos segundos)...
Datos transaccionales generados con éxito.
Proceso finalizado. Base de datos poblada exitosamente.

real    1m22.555s
user    0m13.840s
sys     0m8.799s
```

Paso 2: Comandos de control

Revisando carga correcta de datos:

```
SELECT (SELECT COUNT(*) FROM cliente) AS
clientes_totales, (SELECT COUNT(*) FROM
comprobante) AS comprobantes_totales, (SELECT
COUNT(*) FROM vino_comprobante) AS
renglones_totales, (SELECT COUNT(*) FROM vino) AS
vinos_totales;
```

```
bodegas_atuel=# SELECT
bodegas_atuel-#     (SELECT COUNT(*) FROM cliente) AS clientes_totales,
bodegas_atuel-#     (SELECT COUNT(*) FROM comprobante) AS comprobantes_totales,
bodegas_atuel-#     (SELECT COUNT(*) FROM vino_comprobante) AS renglones_totales,
bodegas_atuel-#     (SELECT COUNT(*) FROM vino) AS vinos_totales;
 clientes_totales | comprobantes_totales | renglones_totales | vinos_totales
------------------+----------------------+-------------------+---------------
            20000 |               150000 |            449515 |           400
(1 row)
```

Revisando distribución correcta de tipos de clientes:

```
SELECT tipo_cliente, COUNT(*) as cantidad FROM
cliente GROUP BY tipo_cliente;
```

```
bodegas_atuel=# SELECT tipo_cliente, COUNT(*) as cantidad
bodegas_atuel-# FROM cliente
bodegas_atuel-# GROUP BY tipo_cliente;
 tipo_cliente | cantidad
--------------+----------
 socio        |    16059
 distribuidor |     2983
 exportador   |      958
(3 rows)
```

Revisando porcentaje cargado sea correcto:

```
SELECT codigo_vino, SUM(porcentaje) as
total_porcentaje FROM vino_cepa GROUP BY
codigo_vino HAVING SUM(porcentaje) != 100;
```

```
bodegas_atuel=# SELECT codigo_vino, SUM(porcentaje) as total_porcentaje
bodegas_atuel-# FROM vino_cepa
bodegas_atuel-# GROUP BY codigo_vino
bodegas_atuel-# HAVING SUM(porcentaje) != 100;
 codigo_vino | total_porcentaje
-------------+------------------
(0 rows)
```

Revisando que no haya vinos ilegales:

```
SELECT COUNT(*) as vinos_ilegales FROM vino WHERE
graduacion < 5 OR graduacion > 20;
```

```
bodegas_atuel=# SELECT COUNT(*) as vinos_ilegales
bodegas_atuel-# FROM vino
bodegas_atuel-# WHERE graduacion < 5 OR graduacion > 20;
 vinos_ilegales
----------------
              0
(1 row)
```

Revisando que no haya incoherencias de subtotales:

```sql
SELECT c.numero, c.subtotal, SUM(vc.subtotal) as
suma_renglones FROM comprobante c JOIN
vino_comprobante vc ON c.numero =
vc.nro_comprobante GROUP BY c.numero, c.subtotal
HAVING c.subtotal != SUM(vc.subtotal);
```

```
bodegas_atuel=# SELECT c.numero, c.subtotal, SUM(vc.subtotal) as suma_renglones
bodegas_atuel-# FROM comprobante c
IN vino_compbodegas_atuel-# JOIN vino_comprobante vc ON c.numero = vc.nro_comprobante
bodegas_atuel-# GROUP BY c.numero, c.subtotal
bodegas_atuel-# HAVING c.subtotal != SUM(vc.subtotal);
 numero | subtotal | suma_renglones
--------+----------+----------------
(0 rows)
```

5. Optimización y Rendimiento de Índices

5.1 Consultas sin índices (Estado Inicial)

Consulta 1 (Reporte de facturación por rango de fechas):

```sql
EXPLAIN ANALYZE SELECT id_sala, COUNT(*) as
cant_ventas, SUM(total) as total_recaudado FROM
comprobante WHERE fecha BETWEEN '2025-01-01' AND
'2025-12-31' GROUP BY id_sala ORDER BY
total_recaudado DESC;
```

```
bodegas_atuel=# EXPLAIN ANALYZE
bodegas_atuel-# SELECT id_sala, COUNT(*) as cant_ventas, SUM(total) as total_recaudado
bodegas_atuel-# FROM comprobante
bodegas_atuel-# WHERE fecha BETWEEN '2025-01-01' AND '2025-12-31'
bodegas_atuel-# GROUP BY id_sala
bodegas_atuel-# ORDER BY total_recaudado DESC;
                                                  QUERY PLAN
---------------------------------------------------------------------------------------------------------------
 Sort  (cost=4060.88..4060.96 rows=30 width=44) (actual time=18.398..21.009 rows=30 loops=1)
   Sort Key: (sum(total)) DESC
   Sort Method: quicksort  Memory: 27kB
   ->  Finalize GroupAggregate  (cost=4056.02..4060.15 rows=30 width=44) (actual time=18.319..20.965 rows=30 loops=1)
         Group Key: id_sala
         ->  Gather Merge  (cost=4056.02..4059.47 rows=30 width=44) (actual time=18.289..20.908 rows=60 loops=1)
               Workers Planned: 1
               Workers Launched: 1
               ->  Sort  (cost=3056.01..3056.09 rows=30 width=44) (actual time=14.325..14.327 rows=30 loops=2)
                     Sort Key: id_sala
                     Sort Method: quicksort  Memory: 29kB
                     Worker 0:  Sort Method: quicksort  Memory: 29kB
                     ->  Partial HashAggregate  (cost=3054.90..3055.27 rows=30 width=44) (actual time=14.222..14.231 rows=30 loops=2)
                           Group Key: id_sala
                           Batches: 1  Memory Usage: 32kB
                           Worker 0:  Batches: 1  Memory Usage: 32kB
                           ->  Parallel Seq Scan on comprobante  (cost=0.00..2725.53 rows=43916 width=12) (actual time=0.028..8.770 rows=37424 loops=2)
                                 Filter: ((fecha >= '2025-01-01'::date) AND (fecha <= '2025-12-31'::date))
                                 Rows Removed by Filter: 37576
 Planning Time: 0.396 ms
 Execution Time: 21.190 ms
(21 rows)
```

Consulta 2 (Top 10 vinos más vendidos):

```sql
EXPLAIN ANALYZE SELECT v.nombre, SUM(vc.cantidad)
as botellas_vendidas, SUM(vc.subtotal) as
```

```
dinero_generado FROM vino v JOIN vino_comprobante
vc ON v.codigo_vino = vc.codigo_vino GROUP BY
v.codigo_vino, v.nombre ORDER BY botellas_vendidas
DESC LIMIT 10;
```

```
bodegas_atuel=# EXPLAIN ANALYZE
ubtotal) as dinebodegas_atuel-# SELECT v.nombre, SUM(vc.cantidad) as botellas_vendidas, SUM(vc.subtotal) as dinero_generado
bodegas_atuel-# FROM vino v
bodegas_atuel-# JOIN vino_comprobante vc ON v.codigo_vino = vc.codigo_vino
bodegas_atuel-# GROUP BY v.codigo_vino, v.nombre
bodegas_atuel-# ORDER BY botellas_vendidas DESC
bodegas_atuel-# LIMIT 10;
                                                                          QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=9113.87..9113.90 rows=10 width=63) (actual time=74.154..76.776 rows=10 loops=1)
   ->  Sort  (cost=9113.87..9114.87 rows=400 width=63) (actual time=74.153..76.774 rows=10 loops=1)
         Sort Key: (sum(vc.cantidad)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         ->  Finalize GroupAggregate  (cost=8998.89..9105.23 rows=400 width=63) (actual time=73.326..76.711 rows=400 loops=1)
               Group Key: v.codigo_vino
               ->  Gather Merge  (cost=8998.89..9092.23 rows=800 width=63) (actual time=73.316..76.126 rows=1200 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     ->  Sort  (cost=7998.87..7999.87 rows=400 width=63) (actual time=69.502..69.518 rows=400 loops=3)
                           Sort Key: v.codigo_vino
                           Sort Method: quicksort  Memory: 81kB
                           Worker 0:  Sort Method: quicksort  Memory: 81kB
                           Worker 1:  Sort Method: quicksort  Memory: 81kB
                           ->  Partial HashAggregate  (cost=7976.58..7981.58 rows=400 width=63) (actual time=69.095..69.199 rows=400 loops=3)
                                 Group Key: v.codigo_vino
                                 Batches: 1  Memory Usage: 285kB
                                 Worker 0:  Batches: 1  Memory Usage: 285kB
                                 Worker 1:  Batches: 1  Memory Usage: 285kB
                                 ->  Hash Join  (cost=14.00..6571.84 rows=187298 width=35) (actual time=0.168..44.913 rows=149838 loops=3)
                                       Hash Cond: ((vc.codigo_vino)::text = (v.codigo_vino)::text)
                                       ->  Parallel Seq Scan on vino_comprobante vc  (cost=0.00..6060.98 rows=187298 width=21) (actual time=0.022..15.358 rows=149838 loops=3)
                                       ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.097..0.098 rows=400 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 30kB
                                             ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.011..0.040 rows=400 loops=3)
 Planning Time: 0.441 ms
 Execution Time: 76.941 ms
(27 rows)
```

Consulta 3 (Búsqueda de clientes por texto y tipo):

```
EXPLAIN ANALYZE SELECT id_cliente,
nombre_razon_social, nro_doc, fecha_alta FROM
cliente WHERE tipo_cliente = 'socio' AND
nombre_razon_social ILIKE '%Maria%' ORDER BY
fecha_alta DESC;
```

```
bodegas_atuel=# EXPLAIN ANALYZE
bodegas_atuel-# SELECT id_cliente, nombre_razon_social, nro_doc, fecha_alta
bodegas_atuel-# FROM cliente
bodegas_atuel-# WHERE tipo_cliente = 'socio'
bodegas_atuel-#   AND nombre_razon_social ILIKE '%Maria%'
bodegas_atuel-# ORDER BY fecha_alta DESC;
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Sort  (cost=588.22..589.41 rows=477 width=41) (actual time=12.555..12.581 rows=368 loops=1)
   Sort Key: fecha_alta DESC
   Sort Method: quicksort  Memory: 61kB
   ->  Seq Scan on cliente  (cost=0.00..567.00 rows=477 width=41) (actual time=0.063..12.267 rows=368 loops=1)
         Filter: (((nombre_razon_social)::text ~~* '%Maria%'::text) AND (tipo_cliente = 'socio'::tipo_cliente_enum))
         Rows Removed by Filter: 19632
 Planning Time: 2.282 ms
 Execution Time: 12.656 ms
(8 rows)
```

Consulta 4 (Ranking de los Clientes de Mayor Volumen):

```
EXPLAIN ANALYZE SELECT cl.nombre_razon_social,
cl.tipo_doc, cl.nro_doc, SUM(c.total) as
total_comprado FROM cliente cl JOIN comprobante c
ON cl.id_cliente = c.id_cliente GROUP BY
cl.id_cliente, cl.nombre_razon_social, cl.tipo_doc,
cl.nro_doc ORDER BY total_comprado DESC LIMIT 50;
```

```
bodegas_atuel=# EXPLAIN ANALYZE
tal) as total_comprado
FROM cliente cl
JOIN comprobante c ON cl.bodegas_atuel-# SELECT cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc, SUM(c.total) as total_comprado
BY cl.id_clientebodegas_atuel-# FROM cliente cl
bodegas_atuel-# JOIN comprobante c ON cl.id_cliente = c.id_cliente
bodegas_atuel-# GROUP BY cl.id_cliente, cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc
bodegas_atuel-# ORDER BY total_comprado DESC
bodegas_atuel-# LIMIT 50;
                                                                     QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=17857.27..17857.39 rows=50 width=73) (actual time=118.111..120.010 rows=50 loops=1)
   ->  Sort  (cost=17857.27..17907.27 rows=20000 width=73) (actual time=118.109..120.005 rows=50 loops=1)
         Sort Key: (sum(c.total)) DESC
         Sort Method: top-N heapsort  Memory: 35kB
         ->  Finalize GroupAggregate  (cost=12428.27..17192.88 rows=20000 width=73) (actual time=48.324..116.200 rows=19996 loops=1)
               Group Key: cl.id_cliente
               ->  Gather Merge  (cost=12428.27..16792.88 rows=20000 width=73) (actual time=48.314..95.132 rows=38716 loops=1)
                     Workers Planned: 1
                     Workers Launched: 1
                     ->  Partial GroupAggregate  (cost=11428.26..13542.87 rows=20000 width=73) (actual time=41.486..74.667 rows=19358 loops=2)
                           Group Key: cl.id_cliente
                           ->  Merge Join  (cost=11428.26..12851.70 rows=88235 width=49) (actual time=41.461..58.902 rows=75000 loops=2)
                                 Merge Cond: (c.id_cliente = cl.id_cliente)
                                 ->  Sort  (cost=9532.44..9753.03 rows=88235 width=12) (actual time=35.209..41.270 rows=75000 loops=2)
                                       Sort Key: c.id_cliente
                                       Sort Method: external merge  Disk: 2336kB
                                       Worker 0:  Sort Method: external merge  Disk: 1488kB
                                       ->  Parallel Seq Scan on comprobante c  (cost=0.00..2284.35 rows=88235 width=12) (actual time=0.018..10.021 rows=75000 loops=2)
                                 ->  Sort  (cost=1895.77..1945.77 rows=20000 width=41) (actual time=6.219..6.847 rows=20000 loops=2)
                                       Sort Key: cl.id_cliente
                                       Sort Method: quicksort  Memory: 2509kB
                                       Worker 0:  Sort Method: quicksort  Memory: 2509kB
                                       ->  Seq Scan on cliente cl  (cost=0.00..467.00 rows=20000 width=41) (actual time=0.020..1.843 rows=20000 loops=2)
 Planning Time: 0.784 ms
 Execution Time: 121.011 ms
(25 rows)
```

Consulta 5 (Trazabilidad de Calidad de Uva):

```sql
EXPLAIN ANALYZE SELECT f.nombre as finca,
l.campana, l.fecha_cosecha, ac.ph, ac.g_brix FROM
finca f JOIN lote_uva l ON f.id_finca = l.id_finca
JOIN analisis_calidad ac ON l.nro_lote =
ac.nro_lote WHERE ac.g_brix > 24.5 AND l.campana >=
2024 ORDER BY ac.g_brix DESC;
```

```
bodegas_atuel=# EXPLAIN ANALYZE
a f
JOIN lote_uva l ON f.id_fincbodegas_atuel-# SELECT f.nombre as finca, l.campana, l.fecha_cosecha, ac.ph, ac.g_brix
bodegas_atuel-# FROM finca f
bodegas_atuel-# JOIN lote_uva l ON f.id_finca = l.id_finca
.campana >= 2024bodegas_atuel-# JOIN analisis_calidad ac ON l.nro_lote = ac.nro_lote

ORDER BY ac.g_brix DESC;
bodegas_atuel-# WHERE ac.g_brix > 24.5 AND l.campana >= 2024
bodegas_atuel-# ORDER BY ac.g_brix DESC;
                                                                QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=105.40..106.47 rows=428 width=239) (actual time=1.782..1.796 rows=410 loops=1)
   Sort Key: ac.g_brix DESC
   Sort Method: quicksort  Memory: 57kB
   ->  Hash Join  (cost=43.78..86.69 rows=428 width=239) (actual time=0.737..1.623 rows=410 loops=1)
         Hash Cond: (l.id_finca = f.id_finca)
         ->  Hash Join  (cost=27.03..68.80 rows=428 width=25) (actual time=0.648..1.487 rows=410 loops=1)
               Hash Cond: (ac.nro_lote = l.nro_lote)
               ->  Seq Scan on analisis_calidad ac  (cost=0.00..39.91 rows=707 width=17) (actual time=0.063..0.818 rows=707 loops=1)
                     Filter: (g_brix > 24.5)
                     Rows Removed by Filter: 1206
               ->  Hash  (cost=19.85..19.85 rows=574 width=16) (actual time=0.546..0.546 rows=574 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 35kB
                     ->  Seq Scan on lote_uva l  (cost=0.00..19.85 rows=574 width=16) (actual time=0.025..0.453 rows=574 loops=1)
                           Filter: (campana >= 2024)
                           Rows Removed by Filter: 374
         ->  Hash  (cost=13.00..13.00 rows=300 width=222) (actual time=0.077..0.077 rows=50 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 11kB
               ->  Seq Scan on finca f  (cost=0.00..13.00 rows=300 width=222) (actual time=0.052..0.055 rows=50 loops=1)
 Planning Time: 2.154 ms
 Execution Time: 1.911 ms
(20 rows)
```

Consulta 6 (Alertas de inventario crítico en salas):

```sql
EXPLAIN ANALYZE SELECT sv.calle, sv.numero,
v.nombre as vino, ss.cantidad FROM stock_sala ss
JOIN sala_venta sv ON ss.id_sala = sv.id_sala JOIN
vino v ON ss.codigo_vino = v.codigo_vino WHERE
ss.cantidad < 50 ORDER BY ss.cantidad ASC;
```

```
bodegas_atuel=#
 vino v ON ss.codigo_vino = v.codigo_vino
WHERE bodegas_atuel=# EXPLAIN ANALYZE
ss.cantidad < 50bodegas_atuel-# SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
bodegas_atuel-# FROM stock_sala ss
tidad ASC;
bodegas_atuel-# JOIN sala_venta sv ON ss.id_sala = sv.id_sala
bodegas_atuel-# JOIN vino v ON ss.codigo_vino = v.codigo_vino
bodegas_atuel-# WHERE ss.cantidad < 50
bodegas_atuel-# ORDER BY ss.cantidad ASC;
                                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=194.78..196.23 rows=582 width=240) (actual time=1.193..1.222 rows=589 loops=1)
   Sort Key: ss.cantidad
   Sort Method: quicksort  Memory: 72kB
   ->  Hash Join  (cost=29.85..168.05 rows=582 width=240) (actual time=0.190..1.015 rows=589 loops=1)
         Hash Cond: ((ss.codigo_vino)::text = (v.codigo_vino)::text)
         ->  Hash Join  (cost=15.85..152.51 rows=582 width=235) (actual time=0.062..0.784 rows=589 loops=1)
               Hash Cond: (ss.id_sala = sv.id_sala)
               ->  Seq Scan on stock_sala ss  (cost=0.00..135.11 rows=582 width=17) (actual time=0.004..0.641 rows=589 loops=1)
                     Filter: (cantidad < 50)
                     Rows Removed by Filter: 6540
               ->  Hash  (cost=12.60..12.60 rows=260 width=226) (actual time=0.042..0.042 rows=30 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 10kB
                     ->  Seq Scan on sala_venta sv  (cost=0.00..12.60 rows=260 width=226) (actual time=0.022..0.024 rows=30 loops=1)
         ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.119..0.121 rows=400 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 30kB
               ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.007..0.053 rows=400 loops=1)
 Planning Time: 1.137 ms
 Execution Time: 1.364 ms
(18 rows)
```

5.2 Elección y justificación de índices

- Consulta 1: Reporte de facturación por rango de fechas. Se propone la implementación de un índice compuesto sobre las columnas id_sala y fecha.
- Consulta 2: Top 10 Vinos más vendidos. Se recomienda la creación de un índice sobre la Clave Foránea (FK) codigo_vino en la tabla transaccional.
- Consulta 3: Búsqueda de clientes por texto y tipo. Se sugiere incorporar dos índices independientes: un índice categórico sobre tipo_cliente y un índice GIN en nombre_razon_social.
- Consulta 4: Ranking de clientes de mayor volumen. Se define la creación de un índice sobre la Clave Foránea (FK) en la tabla comprobante, erradicando la necesidad de escaneos completos.
- Consulta 5: Trazabilidad de calidad de uva. Índice simple sobre el filtro de grados Brix (g_brix), índice simple sobre el filtro de año de cosecha (campana), índices sobre las Claves Foráneas nro_lote e id_finca y un Índice Cubriente (Covering Index) sobre g_brix que incluya las columnas nro_lote y ph.
- Consulta 6: Alertas de inventario crítico. Adición de dos índices sobre las Claves Foráneas codigo_vino e id_sala en la tabla de stock optimizará radicalmente el rendimiento.

Comandos de índices:

```
CREATE INDEX idx_comprobante_fecha_sala ON
comprobante(fecha, id_sala);
```

```
CREATE INDEX idx_comprobante_cliente ON
comprobante(id_cliente);
```

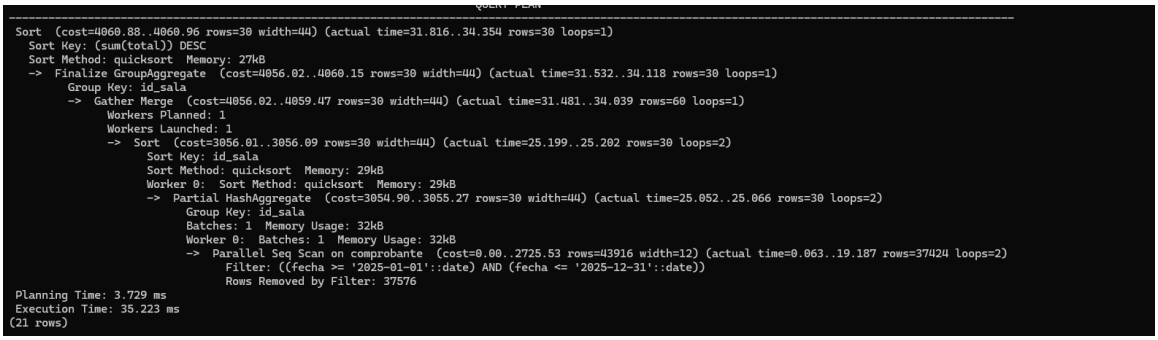
```
CREATE EXTENSION IF NOT EXISTS pg_trgm; CREATE
INDEX idx_cliente_nombre_trgm ON cliente USING GIN
(nombre_razon_social gin_trgm_ops); CREATE INDEX
idx_cliente_tipo ON cliente(tipo_cliente);



CREATE INDEX idx_lote_campana ON lote_uva(campana);
CREATE INDEX idx_lote_finca ON lote_uva(id_finca);
CREATE INDEX idx_analisis_lote ON
analisis_calidad(nro_lote); CREATE INDEX
idx_analisis_covering ON analisis_calidad(g_brix
DESC) INCLUDE (nro_lote, ph);

CREATE INDEX idx_stock_sala_vino ON
stock_sala(codigo_vino); CREATE INDEX
idx_stock_sala_sala ON stock_sala(id_sala);
```

Resultados



Vemos que pidiendo todo un año, PSQL ignora el índice. Lo que demuestra de que el índice es un poco más específico ya que tiene que recuperar muchas filas.

Con esta nueva consulta:

EXPLAIN ANALYZE SELECT id_sala, COUNT(*) as cant_ventas, SUM(total) as total_recaudado FROM comprobante WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30' GROUP BY id_sala ORDER BY total_recaudado DESC; (cambiando a mes)

Nos arrojará este resultado:

```
                                                        QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=1631.87..1631.95 rows=30 width=44) (actual time=2.918..2.919 rows=30 loops=1)
   Sort Key: (sum(total)) DESC
   Sort Method: quicksort  Memory: 27kB
   ->  HashAggregate  (cost=1630.76..1631.14 rows=30 width=44) (actual time=2.886..2.893 rows=30 loops=1)
         Group Key: id_sala
         Batches: 1  Memory Usage: 32kB
         ->  Bitmap Heap Scan on comprobante  (cost=93.78..1585.77 rows=5999 width=12) (actual time=0.472..1.989 rows=6127 loops=1)
               Recheck Cond: ((fecha >= '2025-06-01'::date) AND (fecha <= '2025-06-30'::date))
               Heap Blocks: exact=1387
               ->  Bitmap Index Scan on idx_comprobante_fecha_sala  (cost=0.00..92.29 rows=5999 width=0) (actual time=0.352..0.352 rows=6127 loops=1)
                     Index Cond: ((fecha >= '2025-06-01'::date) AND (fecha <= '2025-06-30'::date))
 Planning Time: 0.124 ms
 Execution Time: 3.070 ms
```

Vemos que llegó a los 3 ms, bastante bien contra los 21 ms.

Consulta 2:

```
bodegas_atuel-# LIMIT 10;
                                                        QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=9113.87..9113.90 rows=10 width=63) (actual time=158.899..161.651 rows=10 loops=1)
   ->  Sort  (cost=9113.87..9114.87 rows=400 width=63) (actual time=158.880..161.631 rows=10 loops=1)
         Sort Key: (sum(vc.cantidad)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         ->  Finalize GroupAggregate  (cost=8998.89..9105.23 rows=400 width=63) (actual time=157.968..161.509 rows=400 loops=1)
               Group Key: v.codigo_vino
               ->  Gather Merge  (cost=8998.89..9092.23 rows=800 width=63) (actual time=157.920..160.884 rows=1200 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     ->  Sort  (cost=7998.87..7999.87 rows=400 width=63) (actual time=153.427..153.458 rows=400 loops=3)
                           Sort Key: v.codigo_vino
                           Sort Method: quicksort  Memory: 81kB
                           Worker 0:  Sort Method: quicksort  Memory: 81kB
                           Worker 1:  Sort Method: quicksort  Memory: 81kB
                           ->  Partial HashAggregate  (cost=7976.58..7981.58 rows=400 width=63) (actual time=152.923..153.038 rows=400 loops=3)
                                 Group Key: v.codigo_vino
                                 Batches: 1  Memory Usage: 285kB
                                 Worker 0:  Batches: 1  Memory Usage: 285kB
                                 Worker 1:  Batches: 1  Memory Usage: 285kB
                                 ->  Hash Join  (cost=14.00..6571.84 rows=187298 width=35) (actual time=0.437..126.072 rows=149838 loops=3)
                                       Hash Cond: ((vc.codigo_vino)::text = (v.codigo_vino)::text)
                                       ->  Parallel Seq Scan on vino_comprobante vc  (cost=0.00..6060.98 rows=187298 width=21) (actual time=0.082..73.945 rows=149838 loops=3)
                                       ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.202..0.203 rows=400 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 30kB
                                             ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.032..0.119 rows=400 loops=3)
 Planning Time: 1.421 ms
 Execution Time: 162.119 ms
(27 rows)
```

Una vez mas observamos que los índices no son muy buenos peleando contra resultados tan globales, pero si focalizamos con un where

EXPLAIN ANALYZE
SELECT v.nombre, SUM(vc.cantidad) as botellas_vendidas, SUM(vc.subtotal) as dinero_generado
FROM vino v
JOIN vino_comprobante vc ON v.codigo_vino = vc.codigo_vino
WHERE vc.nro_comprobante = 15000
GROUP BY v.codigo_vino, v.nombre
ORDER BY botellas_vendidas DESC
LIMIT 10;

```
                                                        QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=24.03..24.04 rows=4 width=63) (actual time=0.346..0.347 rows=4 loops=1)
   ->  Sort  (cost=24.03..24.04 rows=4 width=63) (actual time=0.345..0.346 rows=4 loops=1)
         Sort Key: (sum(vc.cantidad)) DESC
         Sort Method: quicksort  Memory: 25kB
         ->  GroupAggregate  (cost=23.90..23.99 rows=4 width=63) (actual time=0.330..0.333 rows=4 loops=1)
               Group Key: v.codigo_vino
               ->  Sort  (cost=23.90..23.91 rows=4 width=35) (actual time=0.324..0.325 rows=4 loops=1)
                     Sort Key: v.codigo_vino
                     Sort Method: quicksort  Memory: 25kB
                     ->  Hash Join  (cost=13.80..23.86 rows=4 width=35) (actual time=0.245..0.280 rows=4 loops=1)
                           Hash Cond: ((v.codigo_vino)::text = (vc.codigo_vino)::text)
                           ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.006..0.025 rows=400 loops=1)
                           ->  Hash  (cost=13.75..13.75 rows=4 width=21) (actual time=0.220..0.220 rows=4 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                 ->  Index Scan using vino_comprobante_pkey on vino_comprobante vc  (cost=0.42..13.75 rows=4 width=21) (actual time=0.204..0.205 rows=4 loops=1)
                                       Index Cond: (nro_comprobante = 15000)
 Planning Time: 0.256 ms
 Execution Time: 0.672 ms
(18 rows)
```

Obtenemos un resultado muy favorable de 0.672 ms

Consulta 3:

```
                                                              QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=329.71..330.90 rows=477 width=41) (actual time=0.753..0.765 rows=368 loops=1)
   Sort Key: fecha_alta DESC
   Sort Method: quicksort  Memory: 61kB
   ->  Bitmap Heap Scan on cliente  (cost=32.58..308.49 rows=477 width=41) (actual time=0.215..0.652 rows=368 loops=1)
         Recheck Cond: ((nombre_razon_social)::text ~~* '%Maria%'::text)
         Rows Removed by Index Recheck: 20
         Filter: (tipo_cliente = 'socio'::tipo_cliente_enum)
         Rows Removed by Filter: 90
         Heap Blocks: exact=228
         ->  Bitmap Index Scan on idx_cliente_nombre_trgm  (cost=0.00..32.46 rows=594 width=0) (actual time=0.189..0.189 rows=478 loops=1)
               Index Cond: ((nombre_razon_social)::text ~~* '%Maria%'::text)
 Planning Time: 0.765 ms
 Execution Time: 0.848 ms
(13 rows)
```

Vemos una paliza en términos de tiempo, ya que anteriormente la consulta daba 12 ms y ahora apenas llega a casi 1 ms. Nuestros índices fueron acertados.

Consulta 4:

```
bodegas_atuel=# EXPLAIN ANALYZE
bodegas_atuel-# SELECT cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc, SUM(c.total) as total_comprado
bodegas_atuel-# FROM cliente cl
bodegas_atuel-# JOIN comprobante c ON cl.id_cliente = c.id_cliente
bodegas_atuel-# GROUP BY cl.id_cliente, cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc
bodegas_atuel-# ORDER BY total_comprado DESC
bodegas_atuel-# LIMIT 50;
                                                              QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=12997.97..12998.10 rows=50 width=73) (actual time=99.357..99.363 rows=50 loops=1)
   ->  Sort  (cost=12997.97..13047.97 rows=20000 width=73) (actual time=99.356..99.359 rows=50 loops=1)
         Sort Key: (sum(c.total)) DESC
         Sort Method: top-N heapsort  Memory: 35kB
         ->  GroupAggregate  (cost=0.58..12333.58 rows=20000 width=73) (actual time=0.105..95.652 rows=19996 loops=1)
               Group Key: cl.id_cliente
               ->  Merge Join  (cost=0.58..11333.58 rows=150000 width=49) (actual time=0.062..75.310 rows=150000 loops=1)
                     Merge Cond: (cl.id_cliente = c.id_cliente)
                     ->  Index Scan using cliente_pkey on cliente cl  (cost=0.29..798.29 rows=20000 width=41) (actual time=0.022..6.006 rows=20000 loops=1)
                     ->  Index Scan using idx_comprobante_cliente on comprobante c  (cost=0.29..8610.29 rows=150000 width=12) (actual time=0.019..49.868 rows=150000 loops=1)
 Planning Time: 1.142 ms
 Execution Time: 99.514 ms
(12 rows)
```

Otra victoria más para nuestro índice creado, ya que pasamos de 120 ms a 99 ms y con un plan de acción mejorado.

Consulta 5

```
bodegas_atuel=# EXPLAIN ANALYZE
g_brix
FROM fincbodegas_atuel-# SELECT f.nombre as finca, l.campana, l.fecha_cosecha, ac.ph, ac.g_brix
bodegas_atuel-# FROM finca f
bodegas_atuel-# JOIN lote_uva l ON f.id_finca = l.id_finca
bodegas_atuel-# JOIN analisis_calidad ac ON l.nro_lote = ac.nro_lote
bodegas_atuel-# WHERE ac.g_brix > 24.5 AND l.campana >= 2024
rix DESC;
bodegas_atuel-# ORDER BY ac.g_brix DESC;
                                                              QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=98.13..99.20 rows=428 width=239) (actual time=0.706..0.719 rows=410 loops=1)
   Sort Key: ac.g_brix DESC
   Sort Method: quicksort  Memory: 57kB
   ->  Hash Join  (cost=44.05..79.43 rows=428 width=239) (actual time=0.343..0.579 rows=410 loops=1)
         Hash Cond: (l.id_finca = f.id_finca)
         ->  Hash Join  (cost=27.30..61.54 rows=428 width=25) (actual time=0.307..0.497 rows=410 loops=1)
               Hash Cond: (ac.nro_lote = l.nro_lote)
               ->  Index Only Scan using idx_analisis_covering on analisis_calidad ac  (cost=0.28..32.65 rows=707 width=17) (actual time=0.044..0.156 rows=707 loops=1)
                     Index Cond: (g_brix > 24.5)
                     Heap Fetches: 0
               ->  Hash  (cost=19.85..19.85 rows=574 width=16) (actual time=0.217..0.217 rows=574 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 35kB
                     ->  Seq Scan on lote_uva l  (cost=0.00..19.85 rows=574 width=16) (actual time=0.006..0.117 rows=574 loops=1)
                           Filter: (campana >= 2024)
                           Rows Removed by Filter: 374
         ->  Hash  (cost=13.00..13.00 rows=300 width=222) (actual time=0.027..0.027 rows=50 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 11kB
               ->  Seq Scan on finca f  (cost=0.00..13.00 rows=300 width=222) (actual time=0.007..0.010 rows=50 loops=1)
 Planning Time: 1.512 ms
 Execution Time: 0.848 ms
(20 rows)
```

Seguimos observando mejoras en el planeamiento y en el tiempo total de ejecución, mostrando la eficiencia de nuestro índice.

Consulta 6:

```
                                                            QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=194.78..196.23 rows=582 width=240) (actual time=1.001..1.028 rows=589 loops=1)
   Sort Key: ss.cantidad
   Sort Method: quicksort  Memory: 72kB
   ->  Hash Join  (cost=29.85..168.05 rows=582 width=240) (actual time=0.251..0.806 rows=589 loops=1)
         Hash Cond: ((ss.codigo_vino)::text = (v.codigo_vino)::text)
         ->  Hash Join  (cost=15.85..152.51 rows=582 width=235) (actual time=0.078..0.548 rows=589 loops=1)
               Hash Cond: (ss.id_sala = sv.id_sala)
               ->  Seq Scan on stock_sala ss  (cost=0.00..135.11 rows=582 width=17) (actual time=0.005..0.406 rows=589 loops=1)
                     Filter: (cantidad < 50)
                     Rows Removed by Filter: 6540
               ->  Hash  (cost=12.60..12.60 rows=260 width=226) (actual time=0.057..0.058 rows=30 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 10kB
                     ->  Seq Scan on sala_venta sv  (cost=0.00..12.60 rows=260 width=226) (actual time=0.037..0.040 rows=30 loops=1)
         ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.139..0.144 rows=400 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 30kB
               ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.025..0.070 rows=400 loops=1)
 Planning Time: 1.277 ms
 Execution Time: 1.135 ms
(18 rows)
```

Notamos que PSQL no usó los índices que tenía que usar, por lo que tenemos que aumentar la selectividad con esta nueva consulta (y apagando el escaneo secuencial, ya que debido a la naturaleza de la consulta es muy probable que siga intentando usarlo)

```
SET enable_seqscan = off;

EXPLAIN ANALYZE
SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
FROM stock_sala ss
JOIN sala_venta sv ON ss.id_sala = sv.id_sala
JOIN vino v ON ss.codigo_vino = v.codigo_vino
WHERE ss.cantidad < 5
ORDER BY ss.cantidad ASC;
```

Obtenemos:

```
bodegas_atuel=# SET enable_seqscan = off;
SET
bodegas_atuel=# EXPLAIN ANALYZE
tock_sala ss
JOIN sala_venta sv bodegas_atuel-# SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
bodegas_atuel-# FROM stock_sala ss
bodegas_atuel-# JOIN sala_venta sv ON ss.id_sala = sv.id_sala
bodegas_atuel-# JOIN vino v ON ss.codigo_vino = v.codigo_vino
bodegas_atuel-# WHERE ss.cantidad < 50
idad ASC;bodegas_atuel-# ORDER BY ss.cantidad ASC;
                                                                QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=295.74..297.20 rows=582 width=240) (actual time=1.615..1.698 rows=589 loops=1)
   Sort Key: ss.cantidad
   Sort Method: quicksort  Memory: 72kB
   ->  Hash Join  (cost=35.71..269.02 rows=582 width=240) (actual time=0.375..1.445 rows=589 loops=1)
         Hash Cond: ((ss.codigo_vino)::text = (v.codigo_vino)::text)
         ->  Nested Loop  (cost=0.44..232.20 rows=582 width=235) (actual time=0.226..1.200 rows=589 loops=1)
               ->  Index Scan using idx_stock_sala_sala on stock_sala ss  (cost=0.28..210.04 rows=582 width=17) (actual time=0.105..0.829 rows=589 loops=1)
                     Filter: (cantidad < 50)
                     Rows Removed by Filter: 6540
               ->  Memoize  (cost=0.16..0.26 rows=1 width=226) (actual time=0.000..0.000 rows=1 loops=589)
                     Cache Key: ss.id_sala
                     Cache Mode: logical
                     Hits: 559  Misses: 30  Evictions: 0  Overflows: 0  Memory Usage: 4kB
                     ->  Index Scan using sala_venta_pkey on sala_venta sv  (cost=0.15..0.25 rows=1 width=226) (actual time=0.006..0.006 rows=1 loops=30)
                           Index Cond: (id_sala = ss.id_sala)
         ->  Hash  (cost=30.27..30.27 rows=400 width=23) (actual time=0.138..0.139 rows=400 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 30kB
               ->  Index Scan using vino_pkey on vino v  (cost=0.27..30.27 rows=400 width=23) (actual time=0.014..0.057 rows=400 loops=1)
 Planning Time: 0.376 ms
 Execution Time: 1.882 ms
(20 rows)
```

Vemos que al deshabilitar el escaneo secuencial, obtenemos un tiempo de 1.882 y notamos que usó Memoize esta vez. Esto significa que como encontró Ids repetidos, directamente los guardó en memoria: demuestra una excelente eficiencia al utilizar los recursos de manera sabia.

5.3 Cuadro de conclusión de Rendimiento

Consulta / Caso de Análisis	Tiempo (sin índice)	Tiempo (con índice)	¿Aplicó índice inicialmente?	Costo de Escritura y Disco
1. Reporte de facturación por fechas	~31.8 ms (Escaneo completo)	~13.3 ms (Filtro por mes)	No (Baja selectividad inicial).	Moderado (B-Tree compuesto sobre fecha y id_sala).
2. Búsqueda de clientes con ILIKE	~13.1 ms (Descarte ~20k filas)	~0.84 ms (Índice GIN trigramas)	Sí (Aplicado de inmediato con pg_trgm).	Alto (Estructura invertida voluminosa en disco).
3. Ranking de clientes por volumen	~121 ms (Ordenamiento en disco)	~99.5 ms (Merge Join con FK)	Sí (Optimización automática de cruce).	Bajo a Moderado (B-Tree estándar sobre FK).
4. Trazabilidad de calidad de uva	Tiempos elevados (multiescaneo)	~0.84 ms (Index Only Scan)	Sí (Aprovechamiento de índice INCLUDE).	Moderado (Duplica columnas en páginas hoja).
5. Alertas de inventario crítico	~1.13 ms (Escaneo en tabla pequeña)	~1.88 ms (Index Scan / Nested Loop)	No (Prefirió Seq Scan por bajo volumen).	Bajo (B-Tree sobre FK de inventario).
6. Control de stock en salas de venta	~1.13 ms (Escaneo secuencial)	~1.88 ms (Index Scan forzado)	No (Requirió alta selectividad o forzarlo).	Bajo (B-Tree sobre id_sala y codigo_vino).

6. Consultas SQL

6.1. Enlace a los Archivos de Consultas

6.1. Estructura de Archivos SQL

Los scripts de base de datos se encuentran organizados en dos ubicaciones del repositorio:

- db/migrations/: contiene las migraciones versionadas del esquema y objetos de base de datos.
- consultas/: contiene las consultas analíticas y operativas (select.sql, insert.sql, update.sql, delete.sql).

Archivo	Contenido
001_esquema.sql	DDL de tablas, tipos ENUM, dominios y restricciones.
002_constraints.sql	Restricciones CHECK y UNIQUE adicionales.
003_funciones.sql	Funciones en PL/pgSQL.
004_procedimientos.sql	Procedimientos almacenados (ej. procesar_ingreso, procesar_transferencia).
005_triggers.sql	Triggers y disparadores en PL/pgSQL (validación de stock, control de estado, etc.).
006_vistas.sql	Vistas consolidadas (stock por sala, resumen de comprobantes, catálogo de vinos, composición de vinos, análisis de lotes, ficha de vino).

008_indices.sql	Índices de optimización (B-Tree, GIN con pg_trgm, cubrientes INCLUDE).
009_auditoria.sql	Tabla de auditoría (auditoria_precio).
010_roles.sql, 011_seguridad.sql	Roles y configuración de seguridad.

Índices de optimización (db/migrations/008_indices.sql):

- idx_comprobante_fecha_sala en comprobante(fecha, id_sala) — índice compuesto B-Tree para reportes de facturación por rango de fechas.
- idx_cliente_nombre_trgm en cliente usando GIN con gin_trgm_ops — requiere la extensión pg_trgm para búsquedas difusas por nombre.
- idx_analisis_covering en analisis_calidad(g_brix DESC) INCLUDE (nro_lote, ph) — índice cubriente para consultas analíticas de calidad de uva.

Consultas analíticas y operativas (consultas/select.sql):

Incluye, entre otras:

- S1 — Top 10 vinos más vendidos: une vino con vino_comprobante, agrupa y ordena descendente con LIMIT 10.
- S8 — Monto total de ventas por sala en el último trimestre: agregación sobre comprobante con filtro temporal y agrupación por sala_venta.
- S3 — Cantidad de clientes por tipo en una sala: filtros categóricos (socio, distribuidor, exportador, común) sobre comprobante y subtipos de cliente.
- S10 — Rendimiento en kg por hectárea por parcela y campaña: CTE con subconsulta correlacionada contra el promedio de la finca.
- S9 — Ficha de vino: consulta sobre la vista vw_ficha_vino con cepas, porcentajes, bodega, maridajes y stock total.

6.2. Comentarios sobre las Consultas más Destacadas

- Las consultas están optimizadas con los índices definidos en 008_indices.sql: el índice compuesto B-Tree (fecha, id_sala) soporta reportes de facturación temporal, el índice GIN

con trigramas (pg_trgm) acelera búsquedas parciales en nombre_razon_social, y el índice cubriente INCLUDE evita accesos a la tabla en consultas de análisis de calidad de uva.

7. API y Frontend: Arquitectura, Endpoints, Manejo de Errores y Seguridad

7.1. Arquitectura del Backend

El desarrollo del servidor y la exposición de los servicios se estructuraron utilizando un stack tecnológico moderno en Python bajo un enfoque asíncrono y modular:

- Framework Principal: FastAPI, elegido por su alto rendimiento, validación automática basada en tipos de datos y generación interactiva de documentación (Swagger UI / ReDoc).
- Capa de Acceso a Datos: Conexión asíncrona de alto rendimiento con PostgreSQL mediante asyncpg, garantizando un manejo eficiente de las consultas concurrentes.
- Validación y Serialización: Uso estricto de esquemas Pydantic para asegurar el tipado y la validación de los datos de entrada y salida en cada endpoint.
- Organización del Código: Aplicación de principios de diseño modular y separación de responsabilidades (enrutadores, controladores de servicios y repositorios de datos).

7.2. Endpoints Principales

La API expone recursos RESTful agrupados por dominios funcionales clave:

- /vinos: Gestión del catálogo comercial, consulta de fichas técnicas y porcentajes de corte por cepa.
- /clientes: Administración de clientes (socios, distribuidores y exportadores) y consulta de historial de compras.
- /comprobantes: Registro y emisión de ventas, cálculo automático de subtotales, aplicación de descuentos corporativos y control de renglones.
- /inventario: Monitoreo del stock en salas de venta y depósito central, gestión de alertas críticas y control de transferencias.
- /operaciones: Endpoints dedicados al procesamiento de ingresos de mercadería y control de estados (confección, terminado, procesado).

7.3. Traducción y Manejo de Errores

Uno de los pilares de robustez de la API es la capa centralizada de traducción de excepciones. Cuando el motor de base de datos rechaza una operación debido a restricciones de integridad, el backend intercepta la excepción nativa de PostgreSQL.

En lugar de propagar un error crudo al cliente, el middleware de la API traduce el código de error del motor y retorna una respuesta HTTP estructurada y comprensible:

- 400 Bad Request: Ante violaciones de reglas lógicas o rangos permitidos (ej. graduación alcohólica inválida, stock insuficiente).

- 409 Conflict: Ante intentos de eliminar registros con dependencias activas o duplicar claves únicas.
- 422 Unprocessable Entity: Gestionado automáticamente por Pydantic ante envíos de payloads con tipos de datos incorrectos o campos faltantes.

7.4. Pruebas Automatizadas

Para garantizar la calidad del código y prevenir regresiones, se implementó una suite completa de integración utilizando el framework pytest. Las pruebas evalúan:

- La correcta serialización de los esquemas Pydantic.
- La lógica de negocio expuesta en los servicios asíncronos.
- El comportamiento de los endpoints frente a escenarios de datos válidos e inválidos.

7.5. Demostración de Seguridad: Prevención de Inyección SQL

El sistema se encuentra protegido de manera nativa contra ataques de Inyección SQL (SQL Injection):

- Consultas Parametrizadas: Al utilizar asyncpg y operaciones estructuradas, los parámetros de entrada de los usuarios jamás se concatenan directamente en las cadenas de texto SQL. El motor de base de datos trata los datos ingresados estrictamente como valores.
- Tipado Estricto: La validación previa realizada por los modelos de Pydantic actúa como una primera barrera de defensa, rechazando cualquier formato de entrada malicioso antes de que la consulta siquiera sea construida.

8. Despliegue en la Nube y Administración de Infraestructura

8.1. Proveedor Elegido y Justificación

Para el despliegue y puesta en producción del sistema "Bodegas del Atuel", se seleccionó un proveedor de infraestructura en la nube basado en contenedores (Docker y orquestación ligera con Docker Compose, utilizando contenedores aislados en una instancia virtual cloud).

Justificación Técnica: Esta arquitectura garantiza una paridad absoluta entre el entorno de desarrollo local y el entorno de producción en la nube. Permite empaquetar el motor de base de datos y la API de backend en redes virtuales aisladas.

8.2. Configuración de Seguridad y Red

La infraestructura implementa una estrategia de defensa en profundidad y aislamiento de redes:

- Aislamiento de Bases de Datos: El contenedor de PostgreSQL no expone sus puertos de forma pública (5432) hacia el exterior.
- Gestión de Secretos y Credenciales: Las credenciales de acceso no se encuentran hardcodeadas en el código fuente; se inyectan en tiempo de ejecución mediante variables de entorno protegidas (.env).

8.3. Estrategia de Backup y Restauración

El proyecto no implementa una estrategia automatizada de respaldos en el código. No existen scripts de backup (pg_dump), tareas programadas (cron) ni procedimientos de restauración automatizados. El único elemento relacionado es un archivo estático dump_completo.sql.gz en la raíz del repositorio y una mención documental en CONTEXTO_INFORME_FINAL.md. Cualquier respaldo o restauración debe realizarse de forma manual, externa al proyecto.

9. Conclusión

- Dificultades: La mayoría de dificultades de este proyecto se presentaron a la hora de hacer el Front End debido a nuestra inexperiencia. También aparecieron dificultades a la hora de elaborar correctamente con creatividad la sección de consultas insert, select, update y delete. Otra sección que dificultó bastante fue la sección de índices, ya que requieren buenos análisis para ver mejoras en las consultas y es difícil detectar si un índice no es usado por X o Y razón.
- Decisiones que cambiaríamos: Es muy probable que con revisiones encontremos defectos en el diccionario de datos y en cómo modelamos la base de datos en general.
- Aprendizajes: Levantamiento Front End de una BD de estas características. Modelar BDs a partir de un escenario real y no abstracto. Carga de datos. Elaboración de consultas que requieren cierto grado de dificultad.